Оценка качества логического вывода больших языковых моделей в зависимости от порядка конфликта и уровня информационного шума

Дичковский Владимир Андреевич

Аннотация

Инженерия требований критически зависит от своевременного выявления логических противоречий. В исследовании рассматривается проблема оценки способности больших языковых моделей выявлять логические конфликты в требованиях. Существующие общие наборы данных для логического вывода не ориентированы на контролируемое исследование многокомпонентных конфликтов требований. В данной статье предлагается методология тестирования, генерирующая текстовые наборы данных с контролируемым уровнем информационного шума и заданной кардинальностью минимального невыполнимого подмножества требований (количеством требований, образующих противоречие). Для математической верификации конфликтов применяется SMT-решатель Z3. Введен метод структурного доказательства уникальности конфликта, позволяющий верифицировать наборы данных за линейное время. Архитектура разработанной программы включает строго изолированную среду для нейросимвольного исполнения кода. Результаты эксперимента на базе моделей Qwen2.5, Gemma-4 и GPT-OSS статистически подтверждают превосходство нейросимвольного контура над прямым LLM-подходом в точности локализации ядра противоречия, однако выявляют его критическую уязвимость к сбоям кодогенерации у протестированных моделей. Исходный код и наборы данных доступны по адресу: https://github.com/Volodya1000/LogicMUS-Eval.

1 Введение

Спецификации требований служат фундаментальной основой для проектирования инженерных систем (особенно это актуально для таких областей, как банковская сфера, IT-индустрия и аэрокосмическая отрасль). Значение корректной постановки требований критично, поскольку техническая реализация взаимно невыполнимых задач невозможна.

По мере роста сложности систем возникает риск появления конфликтующих требований. Конфликт — это ситуация, при которой одновременное выполнение набора правил приводит к пустому пространству допустимых решений. Специфика проблемы заключается в том, что требования могут быть выполнимыми по отдельности, но становиться несовместимыми при совместном рассмотрении [1].

Ошибки на этапе составления требований приводят к существенному увеличению затрат на исправление дефектов на поздних стадиях разработки. Практическая значимость автоматического выявления конфликтов высока для разработки аппаратно-программных комплексов. Данная задача сегодня решается как традиционными формальными методами, так и с применением больших языковых моделей (LLM).

2 Предшествующие исследования

2.1 Размер конфликта требований

Традиционные методы выявления конфликтов базируются на попарном сравнении требований. В работах А. Саладо и Р. Нилчиани [2] доказана несостоятельность исключительно попарного подхода для выявления конфликтов, у которых степень больше чем 2. Авторы используют термин «порядок конфликта», который в рамках разрабатываемой методологии операционализируется через кардинальность минимального невыполнимого подмножества требований.

2.2 Ограничения языковых моделей в поиске противоречий

В последние годы большие языковые модели активно применяются для анализа спецификаций требований и выявления их несоответствий. Обширный обзор современных методов представлен в работе Zadenoori et al. [3]. Известны работы, в которых языковая модель применяется напрямую — ей передаётся весь набор требований и запрашивается вывод о наличии противоречий [4].

Существуют также гибридные схемы, где требования предварительно группируются по смысловой близости, после чего модель анализирует каждую группу отдельно [5]; такой приём снижает объём контекста, но не гарантирует, что конфликт между требованиями из разных групп будет обнаружен.

Однако прямой LLM-подход (без использования решателей) сталкивается со снижением точности при увеличении структурной сложности задачи [6]. При анализе требований модели способны выявлять очевидные семантические нестыковки, но могут испытывать серьезные трудности при многошаговом логическом выводе.

2.3 Нейросимвольные подходы

Для компенсации указанных недостатков применяются нейросимвольные подходы. Архитектуры, транслирующие естественный язык в формальное представление для последующей проверки решателями, ранее применялись для общих логических задач [7, 8] и верификации автомобильных спецификаций [9]. Аналогичная гибридная логика используется в системе ALICE [10] и в недавних исследованиях генерации проверяющих программ силами LLM [11].

Такой подход позволяет разделить задачу на две независимые подзадачи:

1. понимание естественного языка и преобразование требований в формализованный вид силами LLM;

2. выполнение логического вывода и поиск противоречий в формальной записи с помощью специализированной автоматической программы.

В данной работе в качестве формального решателя выбран SMT-решатель Z3 [12]. Из его аналогов CVC5, Yices 2 и SMTInterpol первый уступает Z3 в удобстве сопоставления правил, второй не имеет официальной библиотеки в PyPI, а третий ориентирован на Java-экосистему. Ключевое преимущество Z3 — механизм `assert_and_track`, позволяющий напрямую связывать булевы переменные-трекеры с идентификаторами исходных требований без повторного разбора AST. Пакет `z3-solver` позволяет вести всю разработку на Python, без внешних прослоек.

3 Постановка задачи и исследовательская ниша

Существуют общие наборы данных для оценки логического вывода LLM, такие как ReClor [13], LogiQA [14], RuleTaker [15] и FOLIO [16]. Однако они ориентированы преимущественно на задачи классификации логических заключений. Влияние структуры самого противоречия и объема сопутствующего информационного шума на качество вывода моделей в контексте спецификаций остается малоизученным. В рассмотренных работах не найдено экспериментальной постановки, одновременно и строго контролирующей кардинальность невыполнимого подмножества и объем независимого шума.

Цель данного исследования — разработка методологии и эталонного набора данных для систематической оценки способности LLM выявлять конфликты. Поставлены следующие вопросы:

1. Как изменяется точность извлечения ядра конфликта при увеличении его кардинальности?

2. Насколько устойчивы прямой текстовый и нейросимвольный подходы к увеличению объема непротиворечивого шума?

4 Терминология и формальная рамка

В рамках разрабатываемой методологии основным параметром выступает кардинальность минимального невыполнимого подмножества (Minimal Unsatisfiable Subset, MUS), обозначаемая как K.

Пусть $\mathcal{R}$ — множество всех требований в спецификации. Подмножество $\mathcal{R}_{MUS} \subseteq \mathcal{R}$ является минимальным невыполнимым, если выполняются два условия:

$$\mathrm{UNSAT}(\mathcal{R}_{MUS}) \quad \wedge \quad \forall r \in \mathcal{R}_{MUS} : \mathrm{SAT}(\mathcal{R}_{MUS} \setminus \{r\}) \tag{1}$$

где SAT обозначает выполнимость, а UNSAT — невыполнимость. Кардинальность $K = |\mathcal{R}_{MUS}|$ выступает метрикой структурной сложности конфликта.

5 Разработка тестового набора и алгоритм генерации

Разработанный генератор создает наборы данных в формате JSONL, сопровождаемые файлом манифеста (`manifest.json`). Каждый тестовый пример в датасете содержит массив правил, отображение абстрактных переменных на естественный язык, статус выполнимости и ожидаемый эталонный MUS.

Тексты требований формируются на русском языке на основе пула заранее подготовленных шаблонов. Целевые и шумовые требования перемешиваются и подаются LLM с одинаковыми порядковыми идентификаторами $(R_1, R_2, \ldots, R_N)$, скрывая от анализирующей модели разницу между смысловым ядром и информационным шумом.

Каждый сценарий генерируется парами: выполнимый (SAT) и невыполнимый (UNSAT). Архитектура использует единое начальное значение генератора псевдослучайных чисел для пары. Различие логического статуса в паре достигается исключительно за счет наличия или отсутствия замыкающего звена в цепочке противоречия.

6 Виды логических паттернов

Топологическая классификация стратегий генерации соответствует базовым классам связей. Паттерны обобщены для произвольного количества переменных n. Для наглядности

механизмы трансляции формальной логики в естественный язык проиллюстрированы на примерах из банковской сферы, где x_i обозначают факты или условия. Данные паттерны предложены в настоящей статье; список не является исчерпывающим, однако достаточен для оценки. Схематическое представление рассматриваемых типов противоречий приведено на рис. 1.

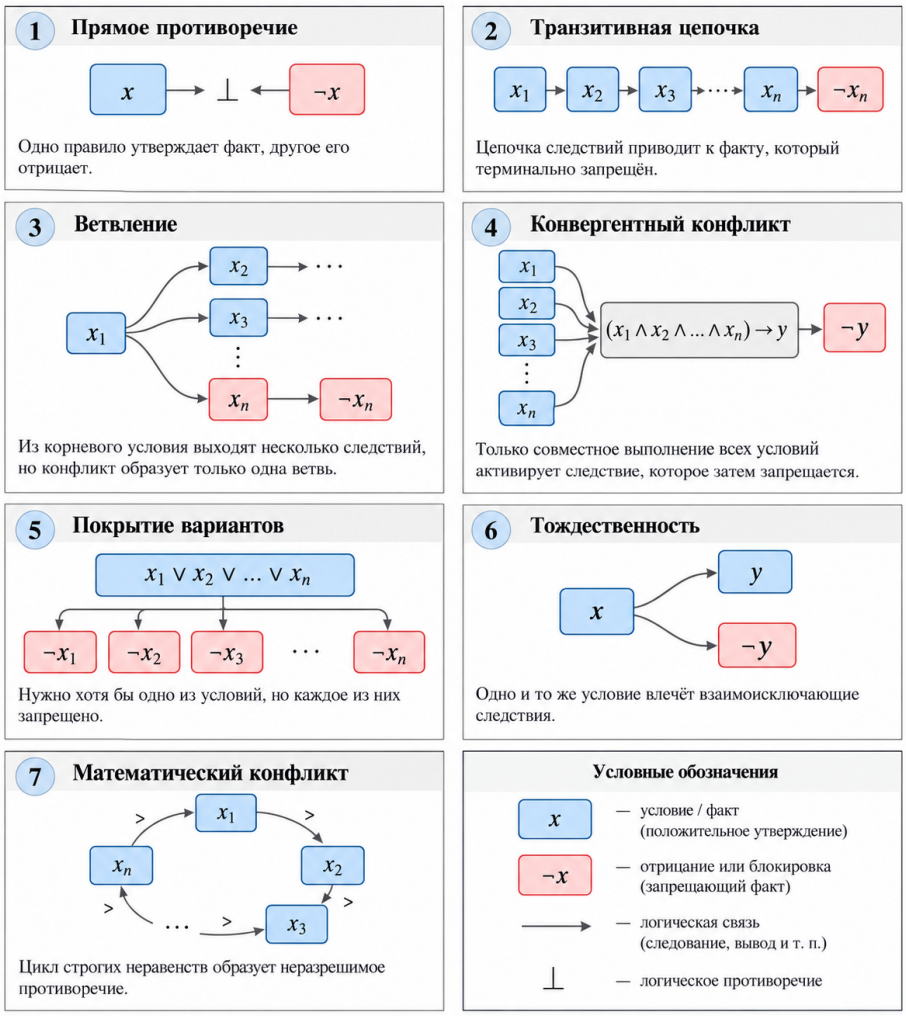


Рис. 1: Виды логических противоречий, используемые при формировании набора данных.

1. **Прямое противоречие.**
   Строго фиксированный размер конфликта $K = 2$. Конфликт возникает между утверждением и его прямым отрицанием, как показано в формуле (2).

$$\text{UNSAT}(\{x, \neg x\}) \tag{2}$$

   Пример:

   R_1: При сумме перевода более 10 000 рублей обязательно применение биометрической верификации.

   R_2: Неверно, что применяется биометрическая верификация.

   Формализация примера: $R_1 : x$, $R_2 : \neg x$.
   Вывод: $x \wedge \neg x \models \bot$.
   Суть конфликта: Утверждения прямо требуют и одновременно запрещают одно и то же действие.

2. **Транзитивная цепочка.**
Масштабируется до n переменных. Итоговая кардинальность ядра составляет $K = n + 1$, что описывается формулой (3).

$$\text{UNSAT}\left(\{x_1, \neg x_n\} \cup \{x_i \to x_{i+1} \mid 1 \leq i \leq n-1\}\right) \tag{3}$$

Пример для $n = 3$ ($K = 4$):

R_1: Клиент успешно вставил карту в банкомат.

R_2: Ввод карты инициирует проверку PIN-кода.

R_3: Успешная проверка PIN-кода открывает доступ к снятию наличных.

R_4: Неверно, что доступ к снятию наличных открывается.

Формализация примера: $R_1 : x_1$, $R_2 : (x_1 \to x_2)$, $R_3 : (x_2 \to x_3)$, $R_4 : \neg x_3$.
Вывод: $x_1 \wedge (x_1 \to x_2) \wedge (x_2 \to x_3) \wedge \neg x_3 \models \bot$.
Суть конфликта: Логическая цепочка требует предоставления услуги, однако терминальное правило R_4 устанавливает запрет.

3. **Ветвление.**
Масштабируется до n переменных. Число генерируемых правил растет с ростом n, однако кардинальность конфликта строго фиксирована: $K = 3$ (корневая переменная, одна импликация и терминальный запрет), согласно формуле (4). Остальные импликации избыточны и не входят в MUS.

$$\text{UNSAT}\left(\{x_1, \neg x_n\} \cup \{x_1 \to x_i \mid 2 \leq i \leq n\}\right) \tag{4}$$

Пример для $n = 3$ ($K = 3$):

R_1: Зафиксирована трансграничная операция.

R_2: Трансграничная операция требует проверки по базе А. (Избыточное правило)

R_3: Трансграничная операция требует проверки по базе Б.

R_4: Неверно, что выполняется проверка по базе Б.

Формализация примера: $R_1 : x_1$, $R_2 : (x_1 \to x_2)$, $R_3 : (x_1 \to x_3)$, $R_4 : \neg x_3$.
Вывод: $x_1 \wedge (x_1 \to x_3) \wedge \neg x_3 \models \bot$.
Суть конфликта: Корневая переменная инициирует проверку (R_3), которая напрямую блокируется (R_4). Правило R_2 не участвует в противоречии.

4. **Конвергентный конфликт.**
Масштабируется до n переменных (антецедентов). Кардинальность ядра составляет $K = n + 2$, что описывается формулой (5).

$$\text{UNSAT}\left(\{x_1, x_2, \ldots, x_n,\ (x_1 \wedge x_2 \wedge \cdots \wedge x_n) \to y,\ \neg y\}\right) \tag{5}$$

Пример для $n = 3$ ($K = 5$):

R_1: Клиент осуществляет оплату корпоративной картой.

R_2: Покупка совершается через офлайн-терминал.

R_3: Операция является трансграничной.

R_4: Одновременное выполнение условий R_1–R_3 требует формирования бумажного акта.

R_5: Неверно, что формируется бумажный акт.

Формализация примера: $R_1 : x_1$, $R_2 : x_2$, $R_3 : x_3$, $R_4 : ((x_1 \wedge x_2 \wedge x_3) \to y)$, $R_5 : \neg y$.
Вывод: $x_1 \wedge x_2 \wedge x_3 \wedge ((x_1 \wedge x_2 \wedge x_3) \to y) \wedge \neg y \models \bot$.
Суть конфликта: Только совместное наличие всех антецедентов активирует следствие, которое затем блокируется терминальным запретом.

5. **Покрытие вариантов.**
Масштабируется до n переменных (альтернатив). Кардинальность ядра составляет $K = n + 1$, что описывается формулой (6).

$$\text{UNSAT}\left(\{x_1 \vee x_2 \vee \cdots \vee x_n,\ \neg x_1,\ \neg x_2,\ \ldots,\ \neg x_n\}\right) \tag{6}$$

Пример для $n = 3$ ($K = 4$):

R_1: Оплата может производиться картами Visa, Mastercard или МИР.

R_2: Неверно, что эквайринг поддерживает карты Visa.

R_3: Неверно, что эквайринг поддерживает карты Mastercard.

R_4: Неверно, что эквайринг поддерживает карты МИР.

Формализация примера: $R_1 : (x_1 \vee x_2 \vee x_3)$, $R_2 : \neg x_1$, $R_3 : \neg x_2$, $R_4 : \neg x_3$.
Вывод: $(x_1 \vee x_2 \vee x_3) \wedge \neg x_1 \wedge \neg x_2 \wedge \neg x_3 \models \bot$.
Суть конфликта: Требуется выполнение хотя бы одного из вариантов, но каждый вариант исключается отдельным требованием.

6. **Тождественность.**
Фиксированный размер $K = 3$, формула (7).

$$\text{UNSAT}\left(\{x, x \to y, x \to \neg y\}\right) \tag{7}$$

Пример:

R_1: Клиент имеет статус VIP.

R_2: Статус VIP гарантирует бесплатное обслуживание.

R_3: Статус VIP обязывает взимать комиссию (неверно, что обслуживание бесплатное).

Формализация примера: $R_1 : x$, $R_2 : (x \to y)$, $R_3 : (x \to \neg y)$.
Вывод: $x \wedge (x \to y) \wedge (x \to \neg y) \models \bot$.
Суть конфликта: Одно и то же условие влечет два взаимоисключающих следствия.

7. **Математический.**
Масштабируется до n переменных ($K = n$), формула (8).

$$\text{UNSAT}\left(\{x_1 > x_2, x_2 > x_3, \ldots, x_n > x_1\}\right) \tag{8}$$

Пример для $n = 3$ ($K = 3$):

R_1: Лимит первой карты строго больше лимита второй.

R_2: Лимит второй карты строго больше лимита третьей.

R_3: Лимит третьей карты строго больше лимита первой.

Формализация примера: $R_1 : (x_1 > x_2)$, $R_2 : (x_2 > x_3)$, $R_3 : (x_3 > x_1)$.
Вывод: $(x_1 > x_2) \wedge (x_2 > x_3) \wedge (x_3 > x_1) \models \bot$.
Суть конфликта: Набор строгих неравенств образует неразрешимый логический цикл.

7 Структурное доказательство уникальности конфликта

Определим граф переменных: вершинами графа выступают правила спецификации, а ребро между двумя вершинами существует тогда и только тогда, когда соответствующие правила используют хотя бы одну общую логическую переменную.

Пусть M — целевое множество правил ожидаемого конфликта, а C — транзитивное замыкание множества M по графу общих переменных. Пусть N — множество добавленных шумовых правил. Алгоритм генерации обеспечивает строгую изоляцию переменных: $V(C) \cap V(N) = \emptyset$. Из этого следует, что между множествами C и N не может возникнуть логического противоречия, так как они оперируют полностью независимыми утверждениями.

Дополнительно в процессе генерации осуществляется проверка, что никакое шумовое правило из N не эквивалентно правилу из M по своей сигнатуре (совпадение типа оператора, переменных и полярности). Внутри самого подмножества C уникальность MUS гарантируется топологической конструкцией используемых паттернов. Данный алгоритм поиска компонент связности позволяет доказать уникальность MUS за линейное время $O(|V| + |E|)$, избегая NP-сложного полного перебора.

8 Контроль лексических утечек

Для исключения возможности определения логического статуса (выполнимо/невыполнимо) по тривиальным лексическим признакам применена векторизация N-грамм (TF-IDF) с классификатором на основе логистической регрессии при кросс-валидации (Stratified Group K-Fold). Установлен жесткий порог: среднее значение F1-меры такого поверхностного классификатора не должно превышать 0,55. Данный барьер исключает возможность успешного угадывания статуса LLM-моделью исключительно на основе статистического распределения слов в тексте. В будущих итерациях протокол будет расширен градиентным бустингом на семантических эмбеддингах.

9 Версионирование набора данных

Для обеспечения полной воспроизводимости экспериментов фиксируются криптографические суммы сгенерированных датасетов и исходного кода (SHA-256), версии библиотек окружения, а также точные значения базового seed.

10 Методология оценки и метрики

10.1 Два подхода к логическому выводу

Оценка LLM проводится по двум независимым стратегиям (рис. 2):

1. **Прямой LLM-подход:** Модели передается системный промпт с инструкцией к пошаговому выводу. Ответ возвращается в формате JSON, включающем статус `is_sat` и массив идентификаторов `conflict_core`.

2. **Нейросимвольный подход (Z3):** Модель транслирует требования в исполняемый код Python. Использование встроенной функции Z3 `assert_and_track` возвращает подмножество конфликтующих утверждений. Поскольку возвращаемое решателем

множество не обязано быть минимальным, оно алгоритмически минимизируется через итеративное удаление элементов для получения строгого MUS.



Рис. 2: Сравнение прямого и нейросимвольного контура обработки требований.

10.2 Обеспечение безопасной среды исполнения LLM-кода

Исполнение сгенерированного кода осуществляется в строго изолированном процессе. Доступные модули ограничены списком (разрешены `z3` и математические функции), встроенная функция `__import__` переопределена для запрета загрузки системных библиотек. Установлен жесткий тайм-аут выполнения в 5.0 с. Инфраструктурные ошибки (например, `SyntaxError`) перехватываются и приравниваются к некорректному ответу.

10.3 Метрики оценки

Замеряются следующие ключевые показатели: точность классификации, доля идеального совпадения ядра, Precision и Recall извлечения элементов конфликта, итоговая MUS F1-мера, доля отказов от ответа и доля ошибок кодогенерации. Дополнительно фиксируется среднее время генерации ответа и объем затраченных токенов на инференс.

11 Дизайн эксперимента

Экспериментальный протокол включает оценку трех современных открытых моделей: `Qwen2.5-Coder-14B-Instruct`, `Gemma-4-E4B` и `GPT-OSS-20B`. Параметры эксперимента строго стандартизированы:

- Температура генерации: 0.0 (для минимизации стохастичности).
- Базовый Seed генератора: 42.
- Баланс выборки: 50% SAT / 50% UNSAT.
- Объем тестов: 100 примеров на каждую контрольную точку сетки.

Исследование разбито на три серии:

1. Оценка влияния информационного шума: сетка из 8 контрольных точек для объема документа N (от 10 до 80 с шагом 10) при зафиксированной кардинальности $K = 3$.

2. Оценка влияния кардинальности: варьирование $K \in \{2, 3, 4, 5, 8, 10, 15\}$ при зафиксированном объеме $N = 40$.

3. Стресс-тестирование на экстремальных конфликтах ($K = 20$).

12 Результаты эксперимента и анализ

В рамках исследования было проведено тестирование по описанному дизайну. Полученные результаты демонстрируют принципиальные различия между способностями моделей и примененными стратегиями логического вывода.

13 Результаты эксперимента и анализ

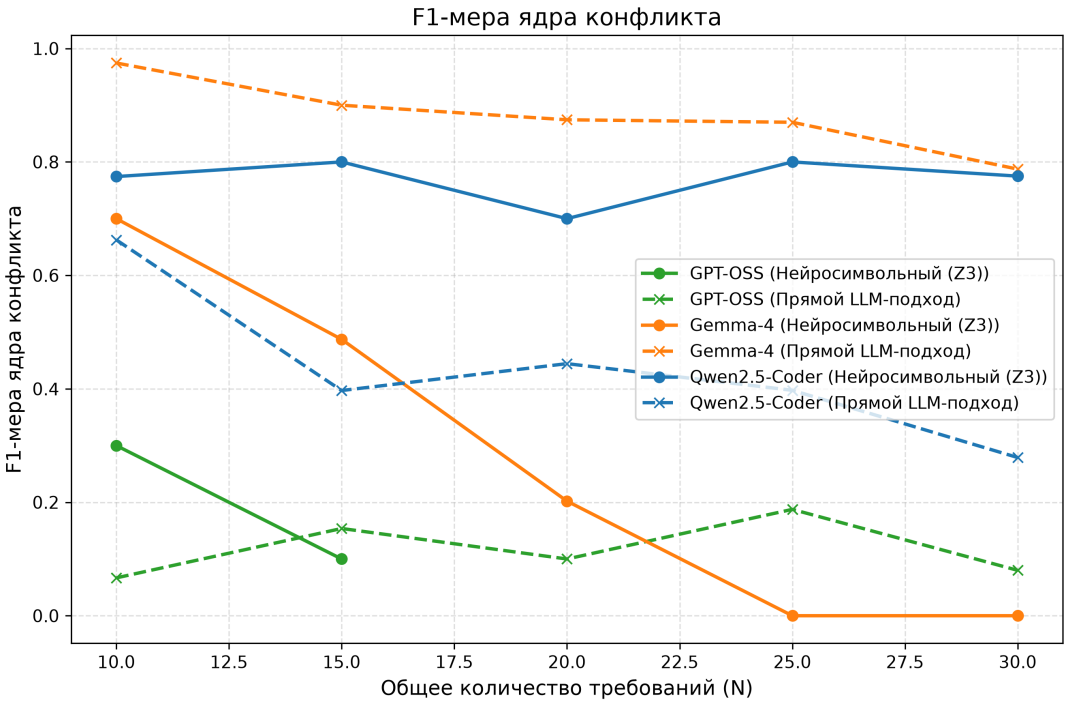
В рамках исследования было проведено тестирование по описанному дизайну. Полученные результаты демонстрируют принципиальные различия между способностями моделей и примененными стратегиями логического вывода.

1. Влияние информационного шума (эксперимент 1). В данной серии кардинальность конфликта зафиксирована на уровне $K = 3$, а варьируется только общее число требований N, что позволяет изолированно оценить устойчивость моделей к росту объёма непротиворечивого контекста. Первая серия экспериментов (при фиксации $K = 3$) показала, что прямой LLM-подход обладает высокой чувствительностью к объему контекста у специализированных моделей. Например, у `Qwen2.5-Coder` показатель F1-меры восстановления ядра деградировал с 0,66 (при $N = 10$) до 0,27 (при $N = 30$). Анализ ответов прямого подхода выявил типичную проблему: модели «размывают» границы ядра, захватывая избыточные непротиворечивые правила или теряя звенья транзитивных цепочек.

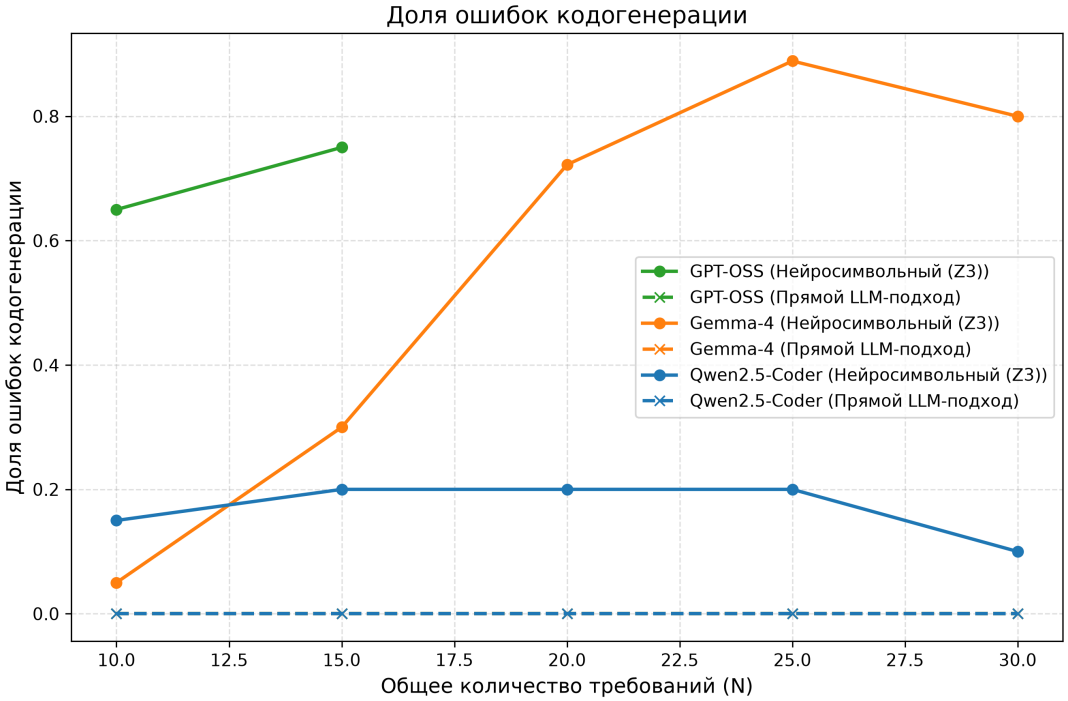
В противовес этому, нейросимвольный подход (решатель Z3) у модели `Qwen2.5-Coder` продемонстрировал высокую устойчивость к зашумлению контекста, стабильно удерживая F1-меру на уровне ∼0,77 (рис. 3a). Модель `Gemma-4-E4B` в прямом подходе показала аномально высокую сопротивляемость шуму, однако полностью провалила нейросимвольный подход при расширении контекста из-за лавинообразного роста сбоев трансляции (рис. 3b).

2. Влияние масштаба конфликта (эксперимент 2). В отличие от первой серии, здесь объём документа зафиксирован на $N = 30$, а варьируется кардинальность ядра K, что позволяет оценить, как рост числа взаимосвязанных требований влияет на точность извлечения минимального конфликта. Результаты второй серии ($N = 30$, варьирование K) доказали несостоятельность прямого текстового вывода для определения точного (минимального) состава сложных противоречий. Для порядков $K \geq 3$ показатель Exact Match (доля случаев, когда модель извлекла идеальный MUS без единой ошибки) у `Qwen2.5-Coder` в прямом подходе обнулился (0,0).

Нейросимвольный контур этой же модели, опирающийся на алгоритмическую минимизацию средствами SMT-решателя, сохранил показатель Exact Match на уровне 0,7–0,8 вплоть до конфликтов с ядром из 7 требований, начав снижаться лишь при $K = 10$ (рис. 4a).

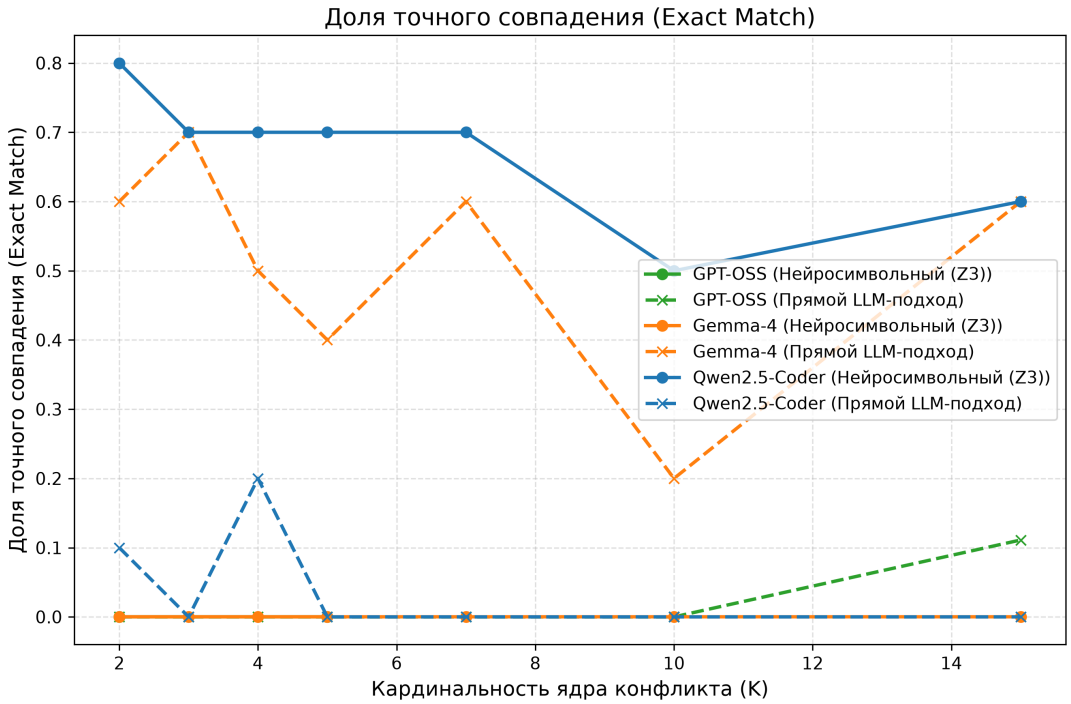


(a) Метрика F1-меры ядра конфликта

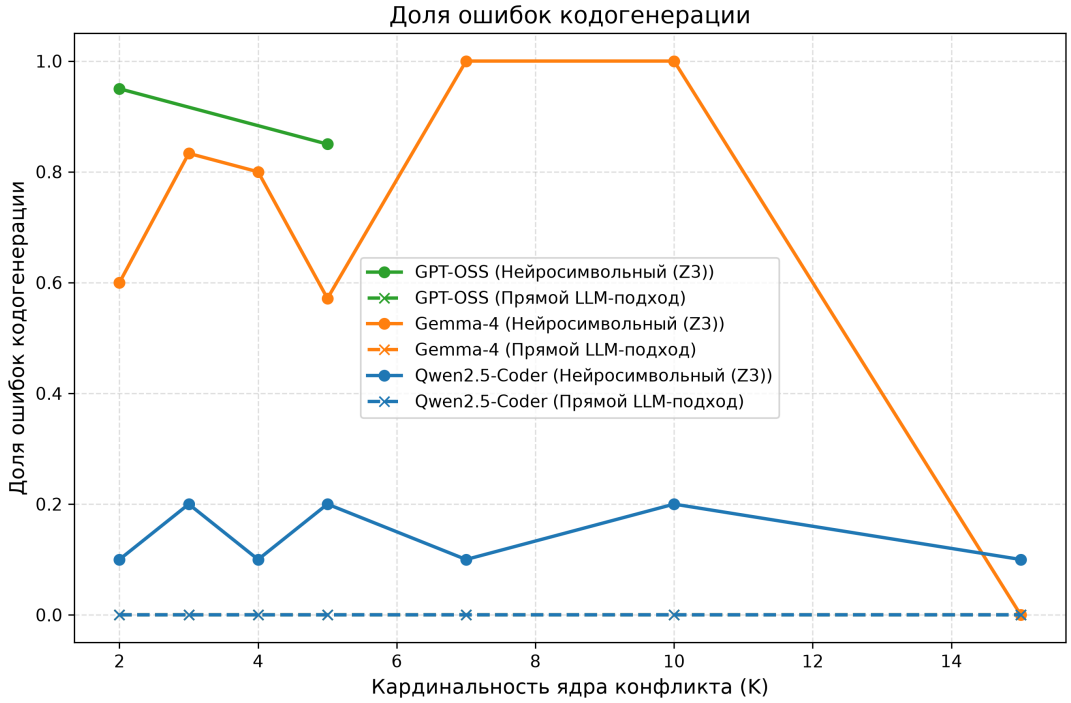


(b) Доля ошибок кодогенерации (отказы Z3)

Рис. 3: Зависимость качества извлечения конфликта от общего числа требований (N)



(a) Доля точного совпадения (Exact Match)



(b) Доля ошибок кодогенерации

Рис. 4: Влияние масштаба конфликта и доля сбоев трансляции при фиксированном $N = 30$

3. Инфраструктурные барьеры нейросимвольного подхода. Данная серия не вводит новых контрольных точек, а обобщает данные первых двух экспериментов с точки зрения доли отказов кодогенерации и ресурсоёмкости трансляции, показывая, что проигрыш нейросимвольного контура может быть обусловлен не логикой вывода, а сбоями инфраструктуры. Главной уязвимостью гибридной стратегии стала неспособность большинства моделей генерировать синтаксически и семантически корректный код на языке Python для API Z3. Модель `GPT-OSS-20B` продемонстрировала долю отказов (Infrastructure Error Rate) от 65% до 95% на всех выборках. Модель `Gemma-4-E4B` успешно справлялась с короткими документами ($N = 10$), но при $N \geq 20$ частота генерации нерабочего кода достигала 80–100% (как видно на рис. 3b и 4b), что приводило к обнулению ее метрик.

Единственной моделью, показавшей рабочие результаты, стала специализированная `Qwen2.5-Coder`, однако и у нее фоновый уровень ошибок трансляции составлял от 10% до 20%, напрямую снижая итоговую полноту системы. Кроме того, трансляция в код оказалась крайне ресурсоемкой: среднее потребление токенов для Z3 у `Qwen` составляло от 35 до 87 тысяч токенов на конфигурацию, что в 1,5–2 раза превышает затраты прямого подхода, пропорционально увеличивая задержку генерации.

4. Дополнительный анализ типов конфликтов и состава ошибок. Для проверки того, что усреднённые показатели не скрывают различия между структурами конфликтов, был добавлен анализ на уровне отдельных случаев. Метка вида восстанавливается из детерминированного round-robin порядка стратегий генератора; она не извлекается из ответа модели. На рис. 5 показана F1-мера MUS для семи видов конфликтов при сравнении прямого и нейросимвольного подходов. Значения усреднены по моделям и всем доступным конфигурациям соответствующего вида.

Состав ошибок на рис. 6 показывает, что нейросимвольный контур нельзя интерпретировать только через качество логического вывода: существенная часть неуспехов относится к ошибкам генерации или исполнения кода. Взаимоисключающие категории построены с приоритетом “ошибка кодогенерации > неверный SAT/UNSAT > ошибка MUS”; случай с одновременно потерянными и лишними элементами MUS относится к категории потери элементов.

Наконец, рис. 7 объединяет влияние размера конфликта K и общего числа требований N. Пустые ячейки намеренно оставлены пустыми: исходный overnight-прогон содержит разреженную L-образную сетку, поэтому значения между фактически измеренными точками не интерполировались.

14 Заключение

Предложенная методология и метод структурного доказательства уникальности $O(|V| + |E|)$ позволили создать строгий инструмент для оценки качества логического вывода языковых моделей в системной инженерии. Использование решателя Z3 в связке с алгоритмической проверкой минимальности решает математическую проблему точного извлечения минимального невыполнимого подмножества (MUS).

Проведенный эксперимент подтверждает, что нейросимвольный подход обладает решающим преимуществом при строгом выделении конфликтов: в то время как прямой текстовый промптинг теряет границы ядра уже при $K \geq 3$, связка LLM+Z3 позволяет безошибочно извлекать конфликты вплоть до $K = 15$ (при наличии специализированной Coder-модели). Однако на сегодняшний день нейросимвольный контур сдерживается критической уязвимостью — нестабильностью генерации исполняемого кода, на которой неспециализированные модели (`GPT-OSS`, `Gemma-4`) терпят полное поражение при увеличении контекста.

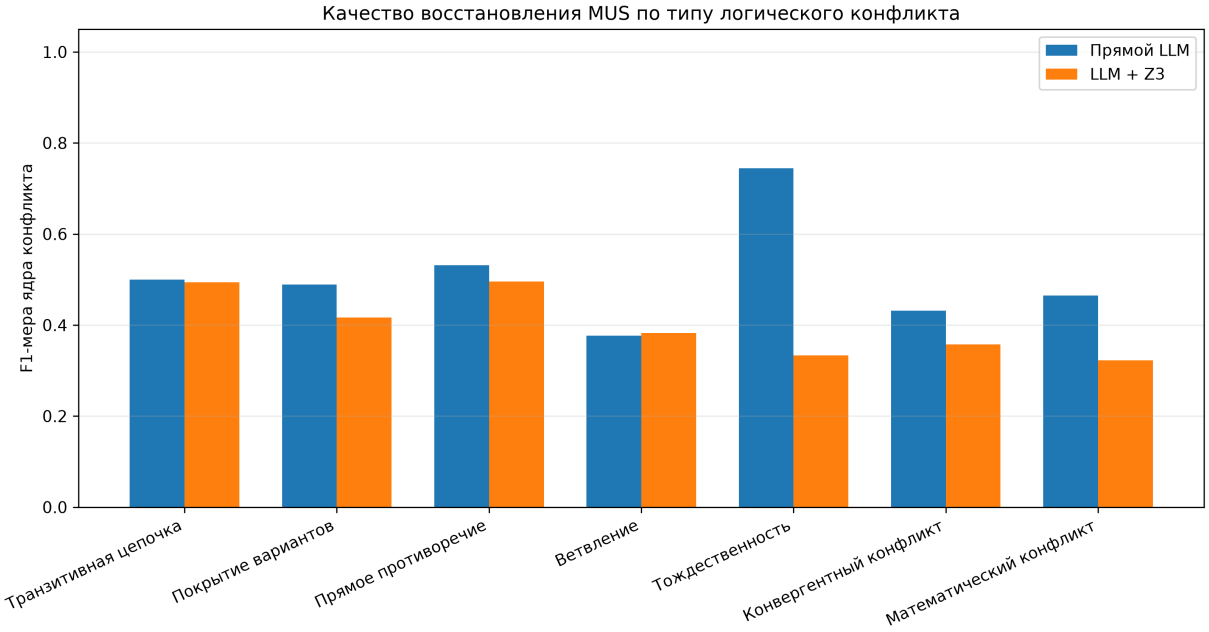


Рис. 5: Средняя F1-мера восстановления MUS по виду логического конфликта.

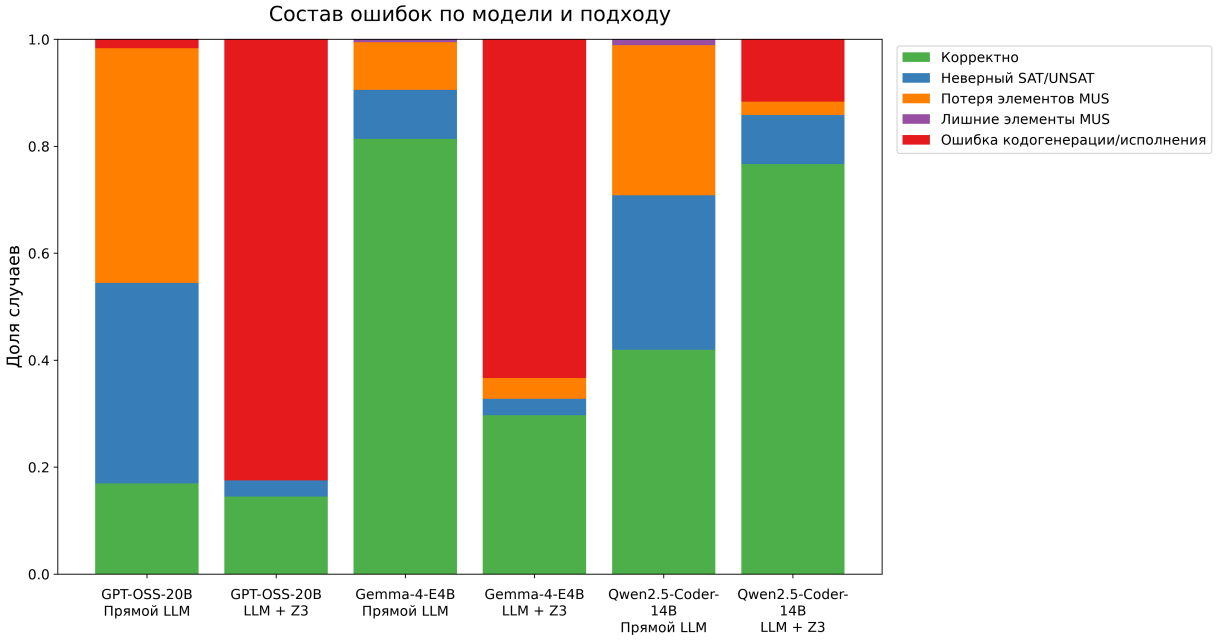


Рис. 6: Состав ошибок по модели и подходу.

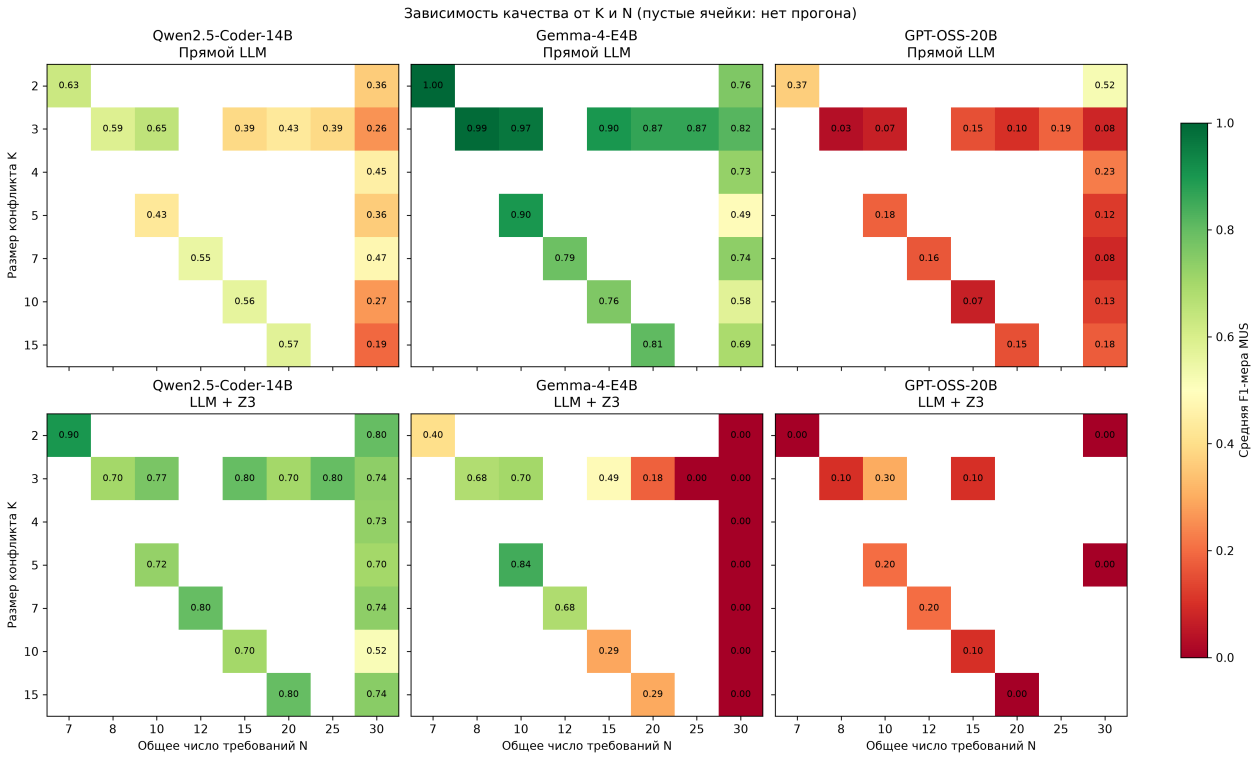


Рис. 7: Зависимость F1-меры MUS от K и N; пустые ячейки означают отсутствие фактического прогона.

Дальнейшая работа будет направлена на замену императивной кодогенерации (Python) на генерацию декларативных спецификаций стандарта `SMT-LIB 2.7`. Ожидается, что этот формат, исключающий управление состоянием и вызовы API, станет более надежным промежуточным слоем между LLM и решателями. Отдельное направление — оценка повышения качества вывода в агентных системах с контурами саморефлексии (Self-Correction) на основе отладочных логов решателя. Для обеспечения воспроизводимости экспериментов генератор и сгенерированные наборы данных опубликованы в открытом доступе.

Список литературы

[1] A. van Lamsweerde, R. Darimont, and E. Letier, "Managing conflicts in goal-driven requirements engineering," *IEEE Transactions on Software Engineering*, vol. 24, no. 11, pp. 908–926, 1998.

[2] A. Salado and R. Nilchiani, "The concept of order of conflict in requirements engineering," *IEEE Systems Journal*, vol. 10, no. 1, pp. 25–35, 2016.

[3] M. A. Zadenoori, J. Dąbrowski, W. Alhoshan, L. Zhao, and A. Ferrari, "Large language models (LLMs) for requirements engineering (RE): A systematic literature review," 2025.

[4] A. Fantechi, S. Gnesi, L. C. Passaro, and L. Semini, "Inconsistency detection in natural language requirements using chatgpt: a preliminary evaluation," in *31st IEEE International Requirements Engineering Conference, RE 2023*, pp. 335–340, IEEE, 2023.

[5] A. Fantechi, S. Gnesi, and L. Semini, "Combining established and emerging techniques to detect inconsistencies in requirements," in *33rd IEEE International Requirements Engineering Conference, RE 2025*, pp. 519–526, IEEE, 2025.

[6] N. Dziri, X. Lu, M. Sclar, X. L. Li, L. Jiang, B. Y. Lin, P. West, C. Bhagavatula, R. Le Bras, J. D. Hwang, S. Sanyal, S. Welleck, X. Ren, A. Ettinger, Z. Harchaoui, and Y. Choi, "Faith and fate: Limits of transformers on compositionality," in *Advances in Neural Information Processing Systems*, vol. 36, pp. 70293–70332, 2023.

[7] T. Olausson, A. Gu, B. Lipkin, C. Zhang, A. Solar-Lezama, J. Tenenbaum, and R. Levy, "LINC: A neurosymbolic approach for logical reasoning by combining language models with first-order logic provers," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, (Singapore), pp. 5153–5176, Association for Computational Linguistics, 2023.

[8] X. Ye, Q. Chen, I. Dillig, and G. Durrett, "SatLM: Satisfiability-aided language models using declarative prompting," in *Advances in Neural Information Processing Systems*, vol. 36, pp. 45548–45580, 2023.

[9] V. Bertram, H. Kausch, E. Kusmenko, H. Nqiri, B. Rumpe, and C. Venhoff, "Leveraging natural language processing for a consistency checking toolchain of automotive requirements," in *2023 IEEE 31st International Requirements Engineering Conference, RE 2023*, pp. 109–119, IEEE, 2023.

[10] A. E. Gärtner and D. Göhlich, "Automated requirement contradiction detection through formal logic and llms," *Automated Software Engineering*, vol. 31, no. 49, 2024.

[11] B. Chen, A. A. Babikian, S. Feng, D. Varró, and G. Mussbacher, "Llm-based satisfiability checking of string requirements by consistent data and checker generation," in *2025 IEEE 33rd International Requirements Engineering Conference, RE 2025*, pp. 308–319, IEEE, 2025.

[12] L. De Moura and N. Bjørner, "Z3: An efficient smt solver," in *International conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, pp. 337–340, Springer, 2008.

[13] W. Yu, Z. Jiang, Y. Dong, and J. Feng, "ReClor: A reading comprehension dataset requiring logical reasoning," in *International Conference on Learning Representations*, 2020.

[14] J. Liu, L. Cui, H. Liu, D. Huang, Y. Wang, and Y. Zhang, "LogiQA: A challenge dataset for machine reading comprehension with logical reasoning," in *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence*, pp. 3622–3628, 2020.

[15] P. Clark, O. Tafjord, and K. Richardson, "Transformers as soft reasoners over language," in *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence*, pp. 3882–3890, 2020.

[16] S. Han, H. Schoelkopf, Y. Zhao, Z. Qi, M. Riddell, W. Zhou, J. Coady, D. Peng, Y. Qiao, L. Benson, L. Sun, A. Wardle-Solano, H. Szabó, E. Zubova, M. Burtell, J. Fan, Y. Liu, B. Wong, M. Sailor, A. Ni, L. Nan, J. Kasai, T. Yu, R. Zhang, A. R. Fabbri, W. M. Kryscinski, S. Yavuz, Y. Liu, X. V. Lin, S. Joty, Y. Zhou, C. Xiong, R. Ying, A. Cohan, and D. Radev, "FOLIO: Natural language reasoning with first-order logic," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, (Miami, Florida, USA), pp. 22017–22031, Association for Computational Linguistics, 2024.